

Föreläsning – 260629 – ASP.NET CORE WEB.API – Loggning

Hur kan man ställa in olika typer av loggning i sitt WEB.API?

Tydlighet i förväntningar på dig som kursdeltagare?

- Det förväntas att du implementerar dessa tips på ert Movie.API **när ni har tid.**
- För att ni ska kunna visa upp några projekt för en framtida arbetsgivare kommer du att ha dels Movie.API, dels slutprojektet.

Det finns flera logging-alternativ i ASP.NET Core, men den vanligaste är att använda den inbyggda **ILogger<T>**-lösningen med t.ex. Console-loggning under utveckling och en mer avancerad provider som **Serilog** i skarpt läge. [learn.microsoft]

Vanliga alternativ

- ILogger<T> med inbyggda providers som Console, Debug och EventSource. [learn.microsoft]
- Serilog för strukturerad loggning, filer, Seq, Elasticsearch eller SQL.
- NLog eller andra tredjepartsproviders.
- Molntjänster som Application Insights för centraliserad övervakning. [learn.microsoft]

Det vanligaste valet

För de flesta ASP.NET Core-projekt är startpunkten:

- ILogger<T> i klasserna.
- Console logging i utveckling.
- **Eventuellt Serilog senare om du vill ha filer eller bättre struktur.**

Man ställer in vilken Log Level man önskar i appsettings.json

appsettings.json använder ASP.NET Core följande standardnivåer för loggning (**Log Levels**), listade från **lägsta till högsta allvarlighetsgrad**:

1. Trace (Nivå 0)

- **Beskrivning:** Mycket detaljerad information. Innehåller ofta känsliga data eller interna systemhändelser.
- **Användning:** Används nästan bara under aktiv felsökning i utvecklingsmiljö.

2. Debug (Nivå 1)

- **Beskrivning:** Användbar information för utvecklare under kodning och testning.
- **Användning:** Visar vad som händer i applikationen steg för steg, men utan den extrema detaljrikedom som Trace ger.

3. Information (Nivå 2)

- **Beskrivning:** Allmänna flödesmeddelanden som visar att appen fungerar som den ska.
- **Användning:** **Standardnivå** för de flesta appar. Loggar när appen startar, vilka API-endpoints som anropas, eller när databasmigrationer körs.

4. Warning (Nivå 3)

- **Beskrivning:** Loggar avvikande eller oväntade händelser som inte kraschar appen, men som bör undersökas.
- **Användning:** Exempelvis misslyckade inloggningsförsök, långsamma databasfrågor eller hanterade undantag (exceptions).

5. Error (Nivå 4)

- **Beskrivning:** Loggar allvarliga fel där en specifik funktion eller ett anrop kraschade.
- **Användning:** När en databasanslutning går ner eller när koden kastar ett oväntat fel (NullReferenceException) som avbryter användarens anrop.

6. Critical (Nivå 5)

- **Beskrivning:** Kritiska systemfel som påverkar hela applikationen eller servern.
- **Användning:** Vid totalt slut på minne (**Out of memory**) eller om databasen är helt permanent oåtkomlig så att appen inte kan starta alls.

7. None (Nivå 6)

- **Beskrivning:** Stänger helt av all loggning.
- **Användning:** Används om man vill tysta en specifik kategori eller ett bibliotek helt och hållet så att det inte skriver något till loggen.

Hur det fungerar i praktiken

När man sätter en **loggnivå** i appsettings.json kommer systemet att logga den nivå **och alla nivåer som är högre (allvarligare)**.

Om du sätter "**Default**": "**Warning**" (**log-level 3**), kommer appen att visa meddelanden för **Warning**, **Error** och **Critical**, men dölja **Information**, **Debug** och **Trace**.

Hur ska man implementera ILogger <T> i en klass eller Controller?

```
public class ProductController : ControllerBase
{
    private readonly ILogger<ProductController> _logger;

    public ProductController(ILogger<ProductController> logger)
    {
        _logger = logger;
    }

    [HttpGet]
    public IActionResult Get()
    {
        _logger.LogInformation("Hämtar alla produkter.");
        var products = new[]
        {
            new { Id = 1, Name = "Keyboard" },
            new { Id = 2, Name = "Mouse" }
        };
    }
}
```

Exempel på nivåer enligt ovanstående **Log Levels**

Vi kan logga på olika nivåer beroende på situation:

- LogTrace
- LogDebug
- LogInformation
- LogWarning
- LogError
- LogCritical

```
_logger.LogInformation("Produkter hämtades.");  
_logger.LogWarning("Ingen produkt hittades.");  
_logger.LogError(ex, "Fel vid hämtning av produkter.");
```

Hur ser det ut om vi aktiverar tex log-level: `_logger.LogDebug`

Appsettings:

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning",  
      "WebAPI_Versioning.Controllers.V1": "Debug"  
    }  
  },  
  "AllowedHosts": "*"   
}
```

Lägg in lämpliga `_logger.LogDebug("");` meddelanden i kontrollern

Studera vad som händer i console-window.